

· · OmenServer · Brainstorming Session

Date : 30 avril 2026 **Sujet :** Transformer un HP Omen 2021 en serveur dédié polyvalent avec panel web **Techniques utilisées :** SCAMPER · Role Playing · First Principles Thinking **Idées générées :**

· Vue d'ensemble du projet

Le concept

Transformer un ancien PC HP Omen (2021) en serveur dédié qui ne fait que ça. Tout est contrôlé depuis un **panel web** accessible depuis n'importe quel appareil (PC, téléphone), inspiré de Minestrator/MineBoard.

Principes fondamentaux

| Principe | Description | |---|---| | **·Dédié** | L'Omen tourne en OS minimal (Linux sans GUI), 100% des ressources au serveur | | **·Modulaire** | Architecture en modules (Jeux, Bots, Fichiers, Média, VPN) avec UI adaptative | | **·Accessible** | Panel web disponible de partout via un domaine perso | | **·Sécurisé** | Authentification + système d'invitations avec rôles/permissions | | **·Multi-support** | Version web pour PC + app mobile pour téléphone | | **·Intelligent** | Diagnostic auto des problèmes, placement intelligent des services |

· Architecture modulaire

Le Hub · Écran principal

Le panel s'ouvre sur un **Hub** central où l'utilisateur choisit un module. Chaque module transforme entièrement l'interface (sidebar, options, actions) pour offrir une expérience simplifiée et dédiée.

Éléments permanents du Hub :

- · Monitoring global (CPU, RAM, température, réseau)
- **·Bouton Kill All** · arrête tous les services d'un coup (urgence)
- · Planification globale inter-modules (calendrier)
- · Centre de notifications

Modules prévus

· Module Jeux

Interface style Minestrator avec :

- Statut en ligne/hors ligne + compteur joueurs
- Boutons Démarrer / Arrêter / Redémarrer
- Sidebar contextuel : Dashboard, Paramètres, Fichiers, Accès, Sauvegardes, Tâches planifiées, Monitoring temps réel, Historique, Joueurs, Versions, Modpacks, Plugins, Mods
- **Intégration CurseForge** catalogue de modpacks, plugins et datapacks installables en un clic
- **Multi-serveurs** : plusieurs serveurs en parallèle (ex: Minecraft + ARK) avec répartition des ressources (slider RAM/CPU)
- Console live et logs temps réel

· Module Bots

Gestion de bots Python personnalisés :

- Liste des bots avec statut (actif/arrêté/erreur)
- **Dashboard personnalisable par type** : graphiques P&L pour trading, liste d'actions pour bots in-game, métriques pour bots d'analyse/prédiction
- **Éditeur de code intégré** : créer, éditer, modifier, supprimer des bots directement depuis le panel
- Logs en temps réel par bot
- Scheduler pour lancer des bots à des heures précises

· Module Fichiers

Cloud personnel style Google Drive / NAS :

- Navigateur de fichiers locaux (stockés sur l'Omen)
- **Intégration Google Drive** : connexion à l'API Google Drive pour naviguer, uploader, télécharger et synchroniser des fichiers entre l'Omen et le Drive
- Règles de sync automatiques (ex: "backup ce dossier vers Drive toutes les nuits")
- Possibilité d'ajouter d'autres services cloud plus tard (Dropbox, OneDrive)

· Module Web

Héberger un site web ou une API personnelle

· Module Média

Serveur de streaming (Plex/Jellyfin) pour films/séries

· Module VPN

Accès sécurisé au réseau domestique à distance

· Système de permissions

Rôles prédéfinis (personnalisables)

Rôle	Permissions par défaut
Spectateur	Voir le statut des services
Joueur	Voir + démarrer/rejoindre un serveur de jeu
Modérateur	Gérer les joueurs (kick/ban), voir les logs
Co-Admin	Presque tout sauf supprimer des services et gérer les permissions

Permissions granulaires

Chaque permission est une checkbox indépendante :

- · Démarrer / · Arrêter / · Redémarrer un service
- · Installer mods/plugins
- · Modifier la configuration
- · Voir la console / les logs
- · Envoyer des commandes console
- · Accéder aux fichiers

Système d'invitation

- L'admin génère un lien d'invitation (`monserveur.fr/invite/abc123`)
- L'invité crée un compte (pseudo + mot de passe)

- L'invité ne voit que les modules autorisés par admin

· Architecture Maître-Agents (Multi-machines)

Concept

L'Omen est le **maître** (cerveau). D'autres PC peuvent être ajoutés comme **agents** (bras). Tout est contrôlé depuis un seul panel.

- Téléphone ······
- PC principal ···
 - ·· OMEN (maître + panel)
 - ··· Module Jeux (ARK · gros serveur)
 - ··· Module Bots (trading)
 - ··· Module Fichiers
 -
 - ·· PC Agent (2ème machine)
 - Module Jeux (Minecraft · petit serveur)
 - Module Bots (scraper)
 - Module Média (Plex)

Placement intelligent

Le panel connaît les specs de chaque machine et recommande automatiquement où placer chaque service :

- Service gourmand (ARK, 16Go RAM) · machine la plus puissante
- Service léger (petit Minecraft vanilla) · machine secondaire
- Possibilité de déplacer un service d'une machine à l'autre

Fonctionnement à distance

Les agents peuvent être sur des réseaux différents (chez un pote, autre ville). Services sensibles au lag (jeux) restent en local, services autonomes (bots, fichiers) peuvent être distants.

·· Fonctionnalités transversales

| Fonctionnalité | Description | ---|---| | · **Monitoring** | CPU, RAM, température, réseau en temps réel | · **Kill All** | Bouton d'urgence sur le hub · tout couper en 1 clic | · **Notifications push** | Alertes sur mobile si crash, surcharge CPU, erreur bot | | · **Diagnostic auto** | Analyse la cause probable d'un problème + suggère un fix | · **Planification globale** | Calendrier inter-modules (ex: Minecraft 14h-22h, bots la nuit) | | · **Thèmes** | Personnalisation de l'apparence du panel | | · **Moniteur réseau** | Surveillance 24/7 de la connexion internet | | · **Wake-on-LAN** | Allumer/éteindre d'autres PC à distance | | · **Sync auto** | Synchronisation de dossiers entre appareils | | · **Souveraineté données** | Tout reste physiquement chez l'utilisateur |

·· Stack technique

| Couche | Technologie | Raison | ---|---|---| | **OS Serveur** | Linux (Debian/Ubuntu Server) sans GUI | Léger, stable, 100% ressources au serveur | | **Backend** | Python + FastAPI | L'utilisateur connaît Python, FastAPI est simple et

performant || **Base de données** SQLite | Fichier simple, pas de serveur DB à installer || **Conteneurisation** | Docker | Isolation des services (chaque serveur de jeu = un conteneur) || **Frontend Web** | HTML/CSS/JS (ou React) | Panel web moderne et responsive || **App Mobile** | PWA ou React Native | Version mobile dédiée **Reverse Proxy** | Nginx | Accès externe via domaine + HTTPS | **Notifications** | Ntfy ou Firebase | Push notifications sur mobile |

· Roadmap de développement

· V1 · Le socle fonctionnel

Objectif : Pouvoir lancer un serveur Minecraft depuis le panel et y accéder de partout.

- [] Installer Linux minimal sur l'Omen (OS sans interface graphique)
 - [] Créer le serveur backend Python (FastAPI + API REST)
 - [] Créer le panel web avec authentification sécurisée
 - [] Wizard de première configuration
 - [] Dashboard principal avec monitoring (CPU, RAM, température)
 - [] Architecture modulaire (système de hub + modules)
 - [] Module Jeux v1 : démarrer/arrêter UN serveur Minecraft
 - [] Configuration accès à distance (Nginx + domaine)
-

· V2 · Enrichissement jeux

Objectif : Un panel de jeux complet comparable à Minestrator.

- [] Multi-serveurs de jeux + répartition des ressources (sliders)
 - [] Intégration API CurseForge (modpacks, plugins, datapacks)
 - [] Console live + logs temps réel
 - [] Système de sauvegardes automatiques
 - [] Tâches planifiées (cron)
 - [] Notifications push mobile
 - [] Système d'invitations + rôles prédéfinis + permissions granulaires
-

· V3 · La polyvalence

Objectif : Le panel gère bien plus que des jeux.

- [] Module Bots : déployer, monitorer, éditer des bots Python
 - [] Dashboards bots customisables par type (trading, gaming, analyse)
 - [] Module Fichiers : cloud personnel + intégration Google Drive (sync, navigation, transferts)
 - [] Diagnostic automatique de problèmes
 - [] Bouton Kill All dans le hub
 - [] Thèmes personnalisables
 - [] Planification globale inter-modules (calendrier)
-

· V4 · Le système

Objectif : Un vrai data center personnel multi-machines.

- [] Architecture Maître-Agents (ajouter des PC supplémentaires)
 - [] Placement intelligent des services selon les specs machines
 - [] Agents distants via Internet (PC chez un pote)
 - [] Module Média (Plex/Jellyfin)
 - [] Module VPN
 - [] App mobile native
 - [] Wake-on-LAN, monitoring réseau, sync auto entre appareils
-

· Inventaire complet des 31 idées

Thème 1 : Infrastructure & Performance

| # | Idée | Source | ---|---|---| | 1 | Dashboard conso perso (CPU, RAM, temp) au lieu de facturation | SCAMPER · Substitute | | 2 | Allocation dynamique des ressources au service actif | SCAMPER · Substitute | 18 | Multi-serveurs avec répartition de ressources (sliders) | Role Playing · Gamer | 25 | Omen en mode OS minimal · 100% ressources au serveur | Role Playing · Admin | 29 | Architecture Maître-Agents multi-machines | SCAMPER · Reverse | 30 | Agents distants via Internet | First Principles | | 31 | Placement intelligent des services selon les specs | First Principles |

Thème 2 : Panel & Interface

| # | Idée | Source | ---|---|---| | 3 | Architecture modulaire avec UI adaptative | SCAMPER · Combine | 10 | Thèmes personnalisables | SCAMPER · Modify | 22 | Kill All dans la zone Hub | Role Playing · Trader | 23 | Double interface Web (PC) + App mobile (téléphone) | Role Playing · Mobile | 26 | Wizard première config + settings ensuite | Role Playing · Admin |

Thème 3 : Module Jeux

| # | Idée | Source | ---|---|---| | 4 | Module Jeux style Minestrator (sidebar complet) | SCAMPER · Combine | 17 | Intégration CurseForge (modpacks, plugins, datapacks) | Role Playing · Gamer |

Thème 4 : Module Bot & Automatisation

| # | Idée | Source | ---|---|---| | 6 | Module Bots / Automatisation Python | SCAMPER · Adapt | 20 | Dashboard bots customisable par type | Role Playing · Trader | 21 | Gestion complète du cycle de vie des bots (créer, éditer, supprimer) | Role Playing · Trader |

Thème 5 : Modules polyvalents

| # | Idée | Source | ---|---|---| | 5 | Modules Fichiers, Web, Média, VPN | SCAMPER · Combine | 13 | Moniteur réseau 24/7 | SCAMPER · Put to Use | 14 | Wake-on-LAN | SCAMPER · Put to Use | 15 | Sync automatique entre appareils | SCAMPER · Put to Use |

Thème 6 : Sécurité & Permissions

| # | Idée | Source | ---|---|---| | 11 | Accès partout via domaine perso | SCAMPER · Reverse | 12 | Souveraineté totale des données | SCAMPER · Reverse | 16 | Système d'invitations avec sous-utilisateurs + permissions | SCAMPER · Eliminate | 19 | Panel de permissions granulaires (checkboxes) | Role Playing · Gamer | 27 | Rôles prédéfinis personnalisables (Spectateur, Joueur, Modérateur, Co-Admin) | Role Playing · Invite |

Thème 7 : Intelligence & Monitoring

· Prochaines étapes

- 1 **Créer un PRD (Product Requirements Document)** Formaliser les exigences du projet
 - 2 **Créer l'architecture technique** · Définir la structure du code, les APIs, la base de données
 - 3 **Préparer l'Open** · Installer Linux Server sur le HP Open
 - 4 **Commencer la V1** · Backend Python + Panel web + Premier serveur Minecraft
-

· Notes de session

Techniques utilisées :

- **SCAMPER** · Déconstruction du concept Minestrator pour créer quelque chose de personnel et polyvalent
- **Role Playing** · 5 personas (Gamer, Trader, Mobile, Admin, Invité) pour découvrir des besoins cachés
- **First Principles** · Tri par faisabilité et priorisation en 4 versions

Découvertes clés :

- La fusion de RAM entre machines est physiquement impossible pour des apps temps réel · l'architecture Maître-Agents est la bonne solution
- Le système de modules avec UI adaptative est le concept central qui rend le projet unique
- Le projet est ambitieux mais découposable en versions gérables, en commençant par un serveur Minecraft basique

Forces créatives de l'utilisateur :

- Vision claire du produit final dès le départ
- Bonnes intuitions sur l'UX (modules qui changent l'interface)
- Questions techniques pertinentes (fusion RAM, réseau distant)
- Pragmatisme sur le scope (savoir ce qu'on enlève, ce qu'on garde)